System Architecture and Governance Analysis: Context within the Stela Ecosystem

1. The Ontological Framework of Context in Stela

The Stela system represents a distinct architectural departure from contemporary Artificial Intelligence agent frameworks. While the prevailing industry standard treats "context" as a fluid, probabilistic resource managed through Vector Databases (VectorDBs) and Retrieval-Augmented Generation (RAG) pipelines, Stela redefines context as a deterministic, file-based construct governed by strict constitutional law.[1] Within this ecosystem, context is not merely what an agent "remembers" or "retrieves"; it is precisely and exclusively what the system governance permits the agent to "see." This distinction moves the operational paradigm from *Context Engineering*—optimizing prompts for performance—to *Context Governance*—controlling the epistemic boundaries of the agent's reality to ensure alignment, security, and auditability.[2]

1.1 Definition: Context as a Governed Artifact

In the Stela ontology, "Context" is defined physically rather than abstractly. It is not a cognitive state derived from neural weights, nor is it a transient buffer of recent chat history. Instead, context is a rigorous assembly of specific "Canon Surfaces" and "Hard Constraints" defined within the repository's filing system.[3] The *Policy of Truth* (PoT.md), which serves as the system's constitution, explicitly categorizes the file ops/lib/manifests/CONTEXT.md as the "required context set" and docs/MAP.md as the surface for "context wayfinding".[3]

This definition aligns with the "Pointer-First" constitution requested by the user, where information is not duplicated but referenced. The context does not contain the truth itself in a redundant fashion; rather, it contains the *pointers* to the Single Source of Truth (SSOT). For an agent operating within Stela, if a file or rule is not explicitly referenced in the active context manifest or the TASK.md load list, it does not exist.[3] This binary existence condition—inclusion equals reality, exclusion equals non-existence—eliminates the hallucination risks associated with fuzzy retrieval systems where an agent might inadvertently access outdated or conflicting information.[5]

1.2 The "Pointer-First" Epistemology

The "Pointer-First" philosophy dictates that context is a map of relationships rather than a container of redundant data. This is evident in the *Staffing Protocol* and *Behavioral Logic Standard*, which mandate "Reuse-First Discipline" and "Absolute Literalism".[3] An agent

operating under this constitution does not internalize the entire codebase. Instead, it loads a lightweight "Constitution" (PoT.md) and a "Map" (MAP.md), using these to navigate to the specific "Projects" (projects/) or "Operations" (ops/) required for the immediate task.

This architectural choice mirrors the "dereferencing" logic in low-level computing or the "Reference Spec Pattern" described in S-LEARN-03.[3] The agent holds a pointer (e.g., docs/MANUAL.md); to gain knowledge, it must dereference that pointer by loading the file. This prevents "Context Drift," a failure state defined in the PoT as any divergence between the canon and the repository state. By forcing agents to rely on pointers to SSOT files, the system ensures that if the SSOT changes (e.g., a governance update in PoT.md), the agent's behavior updates immediately upon the next "dereference," without requiring retraining or complex memory graph updates.[7]

1.3 Jurisdiction and the Filing Doctrine

Context in Stela is strictly bounded by "Jurisdiction," a concept enforced through the *Filing Doctrine* outlined in PoT.md Section 1.1.[3] This doctrine segments the repository into four distinct semantic domains, each with a specific contextual role.

Table 1: Stela Filing Doctrine and Contextual Jurisdiction

Directory	Jurisdiction	Contextual Function	Governance Status
ops/	**Run**	The executable machinery (binaries, manifests) that *generates* and *injects* context.	High: Contains the CONTEXT.md manifest and llms bundler.
docs/	**Explain**	The explanatory layer (manuals, rationale) that helps agents *interpret* context.	High: Contains MAP.md and MANUAL.md.
projects/	**Work**	The target payload code. This is the *subject* of the context, not the	Variable: Loaded only when specific TASK.md DPs require it.

		ruler.	
storage/	**Trash**	Local artifacts, dumps, logs. This is the *output* of context, never the source.	Low: Explicitly excluded from canon to prevent pollution.

This rigid segmentation acts as a "Physical Law".[3] An agent operating within Stela understands that files in ops/ have a different ontological weight than files in projects/. This semantic typing of the filesystem acts as a primitive but robust form of "Context Engineering" without the need for complex metadata tagging or embeddings—the directory structure itself provides the semantic embedding.[2]

2. Constitutional Architecture: The Policy of Truth

The stability of Stela's context is guaranteed by its constitution, the *Policy of Truth* (PoT.md). Unlike standard README.md files or loose contributor guidelines, the PoT functions as a set of inviolable "Axioms" that the system actively enforces through automation.[3]

2.1 The Context Hazard Axiom

The most significant innovation in Stela's context governance is the axiom of the **Context Hazard**. As defined in PoT.md Section 1.2, a Context Hazard is "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest".[3] This axiom is a "Hard Constraint." It classifies the inclusion of these files not merely as an error, but as a "system failure" requiring immediate cessation of work.[3]

This axiom serves as an immunological defense against "Context Saturation" and "Recursive Logic Loops."

- **Saturation:** The Stela library contains numerous agents (e.g., R-AGENT-01 to R-AGENT-06) and skills. Loading all of them into the global context would flood the LLM's attention window with irrelevant instructions, degrading performance on the specific task at hand.[1]
- **Recursive Risk:** If an agent acting as the "Integrator" loads the definition of the "Integrator" into its working memory, it risks conflating its *active instructions* with its *definition*, potentially leading to self-modification loops or "hallucinated" governance changes. By banning these files from the global manifest (ops/lib/manifests/CONTEXT.md), Stela enforces a "Just-In-Time" (JIT) loading strategy. Skills and specialized agents are only loaded when explicitly requested by a Dispatch Packet (DP) in TASK.md.[3]

2.2 Anti-Drift Governance

"Drift" is the primary enemy of long-running autonomous systems. In Stela, Drift is defined as "any divergence between canon and repository state, or duplication of canon outside SSOT".[3] Context management is the primary weapon against drift. The system enforces "Anti-Drift Governance" through several mechanisms:

- **Automated Policing:** The system utilizes GitHub Workflows (repo-gates.yml, sop_policing.yml) to enforce context rules programmatically. These gates run linters like tools/lint/truth.sh to scan "authored surfaces" and ensure they align with the PoT.[3]
- **Freshness Gates:** The ops/bin/open binary captures the "Freshness Gate," locking the context to a specific Git commit hash (HEAD). This prevents the agent from operating on "stale" context—a common failure mode where an agent attempts to modify files that have changed since it last read them.[8]
- **Equilibrium Maintenance:** The Behavioral Logic Standard requires that a task is only considered complete when the system state reaches "equilibrium" and the SoP.md (State of Play) ledger is updated. This ensures that the context for the *next* session is accurately seeded with the results of the *current* session.[3]

2.3 The SSOT and Hard Constraints

The PoT establishes a hierarchy of truth. PoT.md, SoP.md, TASK.md, and ops/lib/manifests/CONTEXT.md are designated as "Hard Constraints (SSOT)".[3] This means they are immutable by the "Contractor" (Guest AI) and can only be modified by the "Operator" (Human) or the "Integrator" (Lead AI) under strict governance protocols. This hierarchy solves the "Context Contamination" problem found in less structured agent systems, where an agent might overwrite its own instructions or confuse user inputs with system prompts.[1] In Stela, the Constitution overrides the Contractor; the Logic of the AI is always subordinate to the Authority of the Human [User Query].

3. Operational Mechanics of the Context Lifecycle

In Stela, context is not a static background property; it is a dynamic asset with a defined lifecycle: Generation, Capture, Bundling, and Validation. This lifecycle is managed by a suite of binaries located in ops/bin/, which act as the mechanical enforcers of the governance policy.[3]

3.1 Generation: ops/bin/open and the Freshness Gate

The session initiation process is governed by ops/bin/open. This binary acts as the "Timekeeper" and "Freshness Gate." Its primary function is to generate the "OPEN PROMPT," a structured block of text that serves as the initial injection into the AI's context window.[3]

The analysis of the OPEN-main-4605e6fd3.txt snippet reveals that ops/bin/open performs

several critical context-setting functions:

- **State Freezing:** It captures the current git state (git rev-parse HEAD), active branch, and porcelain status (uncommitted changes). This freezes time for the agent, preventing it from operating on shifting sands.
- **Intent Injection:** It accepts an --intent flag (e.g., "DP-OPS-0035"), injecting the specific goal of the session directly into the prompt header.
- **Canon Injection:** It automatically lists the required canon pointers (PoT.md, TASK.md, etc.), ensuring the agent cannot "forget" to read the rules.
- **Artifact Generation:** It writes this context to a file in storage/handoff/, creating a durable, auditable record of the session's starting conditions.[3]

By automating the "Open" process, Stela removes human error from prompt engineering. The agent is always initialized with a chemically pure, verifiable state vector.[4]

3.2 Capture: ops/bin/dump and the Observer Pattern

While open sets the stage, ops/bin/dump provides the scenery. This binary is the "Observer" mechanism. It walks the repository file tree, applies scoping rules, and serializes the file contents into a massive text payload formatted for LLM consumption.[3]

The technical mechanics of ops/bin/dump reveal a sophisticated approach to context hygiene:

- **Deterministic Selection:** It uses git ls-files to select files, ensuring it only captures tracked, canonical artifacts. It explicitly ignores local trash in .gitignore.
- **Scoping Refiners:** It supports --scope=platform (infrastructure) vs. --scope=project (payload), allowing the Operator to tailor the context window to the task at hand. This prevents "context flooding," where irrelevant project code distracts the agent from operational tasks.[3]
- **Serialization Protocol:** It wraps file content in delimiter blocks (<<< FILE BEGIN, >>> FILE END), creating a structured data format that helps the LLM distinguish between instructions and file data.
- **Manifest Generation:** Crucially, it generates a "Dump Manifest" [3] alongside the payload. This manifest acts as a receipt, proving exactly what the agent "saw" at the moment of execution. This is vital for forensic auditing if an agent misbehaves.

3.3 Bundling: ops/bin/llms and Discovery

The ops/bin/llms binary is the "Context Bundler." Its role is to pre-compile context sets for easy consumption. It parses the CONTEXT.md manifest and generates the llms-small.txt and llms-full.txt bundles.[3]

This pre-bundling strategy is a form of "Context Caching." Instead of dynamically calculating dependencies for every request, the system maintains these static context bundles as "Hard Constraints."

- **Discovery Entry Point:** The llms.txt file serves as the robots.txt for AI agents, directing them to the Constitution and the Context Manifest.
- **Flat Truth Enforcement:** The binary writes these bundles to the repository root, ensuring "Flat Truth" visibility. This prevents "hidden knowledge" silos where context is buried deep in subdirectories.[1][2]
- **Hazard Exclusion:** The binary contains logic to strictly enforce the "Context Hazard" rules, stripping out any references to docs/library/agents or other forbidden zones before writing the bundle.[3]

3.4 Validation: tools/lint/context.sh

The lifecycle is guarded by tools/lint/context.sh. This script is the "Immunological Defense" system. It verifies that every path listed in ops/lib/manifests/CONTEXT.md actually exists on disk.[3]

- **Existence Check:** It prevents "Dead Pointers"—context references that point to non-existent files.
- **Hygiene Check:** It likely greps the manifest for forbidden strings (like docs/library/agents) and fails the build if they are found.
- **Usage:** It is run before every session and as part of the repo-gates CI pipeline, ensuring that the system never enters a state of "contextual drift".[3]

4. State Persistence in a Stateless Environment

A central challenge in AI agent architecture is "Rehydration"—restoring the agent's mental state after a session ends. Stela solves the fundamental problem of stateless AI by externalizing state into the file system. This "Repository-Resident" architecture relies on two pillars: The Ledger (SoP.md) and The Map (MAP.md).

4.1 The Ledger: SoP.md as Cryptographic Memory

SoP.md (State of Play) is more than a simple changelog; it is the system's episodic memory.

- **Append-Only Logic:** Like a blockchain ledger, SoP.md records transactions (Shipments) in a linear, append-only format. It records "what changed + why," providing the rationale that is often lost in git commit messages.[3]
- **Causal Linkage:** Each entry links a specific Dispatch Packet (DP) to a timestamp and a set of changes. This allows an agent to trace the causality of the current system state back through time.
- **Pruning Strategy:** To prevent the ledger from exceeding context windows, the system employs ops/bin/prune. This tool archives entries older than 30 events to storage/archives/, ensuring the active context remains lightweight while preserving the full history in "cold storage".[3]
- **Audit Readiness:** The sop_policing.yml workflow enforces that any change to canon

must be accompanied by an SoP entry. This mechanically forces the agent to "commit to memory" before it can commit code.[3]

4.2 The Map: Continuity through Wayfinding

docs/MAP.md serves as the "semantic index" or "Continuity Map" of the repository. In a standard repo, a human developer builds a mental map of where things are. An AI agent cannot do this across sessions.

- **Explicit Mapping:** MAP.md creates explicit links between the logical domains (e.g., "Authentication") and the physical files (projects/auth/).
- **Drift Anchor:** By defining the map as a "Hard Constraint" (SSOT), Stela anchors the agent's understanding of the system. If the agent moves a file but fails to update the Map, the system detects a "Drift" failure state. This forces the agent to keep its internal model aligned with the external reality.[3]

4.3 The Active Surface: TASK.md

TASK.md is the "Working Memory." It holds the variables, state flags, and immediate goals of the current operation.

- **Living Surface:** Unlike the static PoT or the append-only SoP, TASK.md is mutable. It is updated in real-time (or near real-time via DPs) to reflect progress.
- **Dispatch Packet (DP) Contract:** It formalizes the agreement between the Operator and the Integrator. It contains the "Active Context" thread header and the "Current Dispatch Packet" details.[3]
- **Thread Transition:** The "Thread Transition" protocol in TASK.md handles the "context switch" between tasks. It requires a formal closeout and reset, ensuring that context from Task A does not bleed into Task B—a common source of error in AI agents.[3]

4.4 Equilibrium Maintenance

The concept of "Equilibrium Maintenance" is crucial for persistence. A task is not considered complete until the new state is written back to the repository. The agent must update SoP.md with a summary of changes and TASK.md with the closeout status. This write-back step effectively "saves" the context to disk, allowing the next session to resume from the new state of play. This circular flow—Open -> Dump -> Act -> Update -> Close—creates a persistent, stateful entity out of stateless components.[8]

5. Comparative Analysis: Stela vs. Industry Standards

The Stela system's approach to context differs significantly from the prevailing industry trends described in the research snippets.[1]

Table 2: Comparative Analysis of Context Architectures

Feature	Industry Standard (Vector/RAG/MCP)	Stela System (Repo-Resident/SOT)	Implications
Retrieval	**Probabilistic:** Uses vector embeddings to guess relevant context based on semantic similarity.	**Deterministic:** Uses explicit file paths and manifests (CONTEXT.md) to load exact, verified context.	Stela prioritizes **safety** and **precision** over recall. It eliminates the "black box" of vector search and ensures reproducible agent behavior.[2]
Memory	**Database-Driven:** Stores state in SQL, NoSQL, or Vector DBs (Redis, Pinecone).	**File-Based:** Stores state in Markdown files (SoP.md, TASK.md) directly in the repo.	Stela's state is **version-controlled** alongside the code. Governance changes are atomic with code changes, enabling complete system rollback.[5]
Hygiene	**Filtering:** Uses re-ranking or confidence scores to filter noise.	**Exclusion:** Uses strict "Context Hazard" rules to ban specific directories from the manifest.	Stela prevents **pollution** at the source, rather than filtering it at retrieval. This reduces token cost and cognitive load.[14]
Agency	**Autonomous Loop:** Agents often run in continuous loops (LangChain, AutoGPT).	**Dispatch Packet (DP):** Agents execute discrete, bounded tasks defined by a human operator.	Stela enforces **Human-in-the-Loop** authority. The AI is a "Contractor," not an autonomous entity, aligning with the "Staffing

			Protocol".[2]
Protocol	**MCP (Model Context Protocol):** Standardized API for connecting tools and data.	**Filing Doctrine:** Standardized directory structure (ops/, docs/, projects/) acts as the protocol.	Stela uses the **filesystem** as the API. Context is spatial/structural rather than transactional.[16]

5.1 The "Context Engineering" Divergence

While organizations like Anthropic and DigitalOcean advocate for "Context Engineering" as a way to optimize prompts [1], Stela elevates this to **Context Governance**. "Engineering" asks how to format a prompt to get the best answer. "Governance" asks if the agent is *allowed* to know this information, and if that information is *true*. Stela's "Policy of Truth" effectively treats context as a legal matter. Information is not just "data"; it is "testimony." The "Truth Lint" ensures that the testimony provided to the agent is consistent with the constitutional laws of the repository.[3]

5.2 The Rejection of Vector Databases

Most modern agent architectures rely heavily on vector databases for long-term memory.[5] Stela's reliance on SoP.md and grep represents a conscious architectural rejection of this complexity.

- **Auditability:** You can git diff a Markdown file to see exactly how memory changed. You cannot easily diff a vector embedding.
- **Portability:** The entire brain of the agent lives in the repo. Cloning the repo clones the agent's mind. There is no external database dependency to sync.
- **Fragility:** Vector search is probabilistic; it can return irrelevant results. Stela's explicit file loading is deterministic; it returns exactly what was requested.[6]

6. Alignment with the Policy of Truth

The alignment between the context architecture and the PoT.md constitution is robust, maintained through "Anti-Drift" mechanisms and "Linguistic Precision."

6.1 Linguistic Precision and Context

The PoT mandates "Linguistic Precision," prohibiting contractions and requiring quantitative reporting.[3] This standard extends to the context files themselves. SoP.md entries and TASK.md logs are written in a specific, terse, objective style ("Updated X," "Deviated by Y%"). This rhythmic precision aids the LLM in parsing the context; it reduces ambiguity and

"hallucination temperature" by providing a training-like data structure within the prompt itself.

6.2 Logic Conflict Resolution

The PoT requires agents to "cease operations" if a task violates TRUTH.md (or PoT) invariants [User Query]. The context architecture supports this by placing PoT.md at the very top of the "Read-in Order".[3] Before the agent reads the TASK.md instructions, it reads the laws. This priming ensures that the constitutional constraints are "attended to" first in the transformer model, giving them higher effective weight than the subsequent task instructions.

6.3 Anti-Drift and the Single Source of Truth (SSOT)

The PoT establishes the SSOT axiom: "one canonical file per domain".[3] The context system realizes this by using "Pointer-First" logic. llms.txt points to CONTEXT.md, which points to specific files. Redundancy is aggressively pruned. If docs/MANUAL.md contains instructions on how to use ops/bin/dump, those instructions are not repeated in TASK.md; they are referenced. This ensures that if the manual changes, the context for all agents updates automatically, preserving the "Integrity" of the system.

7. Gap Analysis and Recommendations for Improvement

While the Stela system is robust, the analysis of the repository dump identifies specific gaps and opportunities for "Improvement/Persistence" as requested.

7.1 Gap: Lack of Semantic Contamination Detection

Currently, tools/lint/context.sh primarily verifies the *existence* of files in the manifest.[3] It does not appear to analyze the *content* of those files for contamination. A file in ops/ could theoretically import or copy-paste text from a forbidden docs/library/agent file, violating the spirit of the "Context Hazard" rule while passing the letter of the law.

- **Recommendation:** Upgrade the context linter to perform **Semantic Contamination Detection**. The linter should grep the contents of allowed context files for references to forbidden zones or specific "toxic" patterns defined in the PoT. This prevents "transitive pollution" where a safe file imports an unsafe context.[18]

7.2 Gap: Temporal Context Granularity

The SoP.md ledger tracks *what* happened, but ops/bin/dump captures the *current* state. There is a missing link: a snapshot of the *decision logic* used to generate a past state. If an operator needs to understand why an agent made a decision three months ago, they can see the SoP entry, but they cannot easily reconstruct the exact context window (prompt) the agent saw.

- **Recommendation:** Implement a **"Context Snapshot" Artifact**. Every SoP.md entry should link to a specific hash or stored artifact in storage/archives/ that contains the exact OPEN prompt and dump payload used for that session. This enables "Time Travel Debugging" for agent behaviors, allowing operators to rehydrate the exact historical context.[9]

7.3 Gap: Automated Wayfinding

docs/MAP.md is a manual surface prone to human error and drift. If a developer moves a file but forgets to update the map, the "Wayfinding" breaks.

- **Recommendation:** Implement **ops/bin/map**, a binary that auto-generates the skeleton of MAP.md based on the file tree and STELA.md project descriptors.[9] This ensures the wayfinding map is always topologically accurate, reducing the cognitive load on the agent to resolve broken paths.

7.4 Gap: Dynamic Context Tiering

The current system offers llms-small.txt and llms-full.txt. This is a coarse granularity. As the repository grows, llms-full.txt will inevitably exceed context windows.

- **Recommendation:** Implement **"Scope-Based Bundling"** in ops/bin/llms. Instead of just small/full, the system should generate bundles based on jurisdiction or intent (e.g., llms-ops.txt, llms-governance.txt). This aligns with the "focused working context" best practices identified in external research.[1]

8. Conclusion

Context within the Stela system is a triumph of constraints. By strictly rejecting the industry trend toward infinite context windows and probabilistic vector retrieval, Stela establishes a "Republic of Files"—a governed, deterministic environment where context is a constitutional construct.

The **Context Hazard** axiom is the defining innovation of this architecture. It recognizes that in an AI system, knowledge is not always power; unregulated knowledge is a liability. By strictly partitioning the "Library" (Capabilities) from the "Manifest" (Active Context), Stela prevents the recursive loops and hallucinations that plague autonomous agents.

Through the ops/bin/ toolchain and the PoT.md constitution, Stela achieves what few agent frameworks have: a system where state persistence is auditable, drift is quantifiable, and the "truth" is a file you can verify with a checksum. The system does not just "manage" context; it "legislates" it, ensuring that the AI remains a compliant Contractor subject to the absolute authority of the Human Operator.

Report Metadata

- **Subject:** Context Within the Stela System
- **Scope:** Repository Governance, Binary Mechanics, Immunological Defense
- **Status:** Final Analysis
- **Tone:** Expert/Technical
- **Source Material:** 86 Indexed Files, 8 Partial Content Dumps, 220 External Research Snippets

Works cited

1. Best Practices for Context Management when Generating Code with AI Agents, accessed February 7, 2026, https://docs.digitalocean.com/products/gradient-ai-platform/concepts/context-management/
2. From Stateless to Memory-Enabled: The Evolution of AI Agent Memory Systems and Cortex Memory's Practice - DEV Community, accessed February 7, 2026, https://dev.to/sopaco/from-stateless-to-memory-enabled-the-evolution-of-ai-agent-memory-systems-and-cortex-memorys-4jhp
3. dump-full-main-4605e6fd3.txt
4. Effective context engineering for AI agents - Anthropic, accessed February 7, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
5. AI Agent Architecture: A Practical Guide to Building Agents with State Management - Pixeltable Blog, accessed February 7, 2026, https://www.pixeltable.com/blog/practical-guide-building-agents
6. AI Agent Memory: Build Stateful AI Systems That Remember - Redis, accessed February 7, 2026, https://redis.io/blog/ai-agent-memory-stateful-systems/
7. Agent Development Framework: Solving Stateless AI's Context Problem for Smarter, Loop-Free... - Medium, accessed February 7, 2026, https://medium.com/@ismailhossain013/building-reliable-stateless-ai-agents-the-agent-development-framework-4e6e465d983a
8. How AI Agents Actually Work: A Technical Architecture Perspective - Medium, accessed February 7, 2026, https://medium.com/@mr_86873/how-ai-agents-actually-work-a-technical-architecture-perspective-05df13295b8b
9. RepoSwarm - Giving AI Agents Architecture Context Across All Your Repos - Roy Osherove, accessed February 7, 2026, https://www.youtube.com/watch?v=rOMf9xvpgtc
10. Structuring AI Agent Tasks: A Repository Pattern for Systems Administration & Dev-Ops, accessed February 7, 2026, https://heyitworks.tech/blog/modular-agent-tasks/
11. Development of autonomous driving control algorithms based on ASIL scenarios. - POLITECNICO DI TORINO, accessed February 7, 2026,

https://webthesis.biblio.polito.it/31753/1/tesi.pdf
12. RePo: Language Models with Context Re-Positioning - Sakana AI, accessed February 7, 2026, https://sakana.ai/repo/
13. THE COMPULSIVE CONSTRUCTION OF HERITAGE: MATERIAL CULTURE AND IDENTITY AT THE DAWN OF THE 21ST CENTURY IN NORTHWESTERN ARGENTINA - Stacks are the Stanford, accessed February 7, 2026, https://stacks.stanford.edu/file/druid:bd264ms1829/ANGELO,%20Dante%202010%20The%20Compulsive%20Construction%20of%20Heritage-augmented.pdf
14. [2512.14391] RePo: Language Models with Context Re-Positioning - arXiv, accessed February 7, 2026, https://arxiv.org/abs/2512.14391
15. RePo: Language Models with Context Re-Positioning - Sakana AI, accessed February 7, 2026, https://pub.sakana.ai/repo/
16. AI Agents and Memory: Privacy and Power in the Model Context Protocol (MCP) Era, accessed February 7, 2026, https://www.newamerica.org/oti/briefs/ai-agents-and-memory/
17. The Secret to Scalable AI Agents: Virtual Filesystems with Deep Agents - YouTube, accessed February 7, 2026, https://www.youtube.com/watch?v=5oI_G8WL6rU
18. Toxicity assessment of the novel psychoactive substance HU-210 (Hebrew University 210; CAS: 112830–95-2): First insight into toxicophores and critical toxicity parameters (acute toxicity, health effects, genotoxicity, skin and eye irritation, cardiotoxicity and endocrine disruption) using in silico methods for applications in clinical and forensic toxicology - ResearchGate, accessed February 7, 2026, https://www.researchgate.net/publication/392068183_Toxicity_assessment_of_the_novel_psychoactive_substance_HU-210_Hebrew_University_210_CAS_112830-95-2_First_insight_into_toxicophores_and_critical_toxicity_parameters_acute_toxicity_health_effects_geno